CISC 451 Assignment 1: Text-to-SQL on Spider with Qwen Models

[Nguyen Nguyen, 20360357]

October 2026

1. Task 1 – RQ1: How do (selected) LLMs perform on cross-domain Text-to-SQL?

Methodology

Models. I evaluate three open-weight Qwen models that cover both required categories (Table 1). Category A (decoder-only, no explicit reasoning) is Qwen2.5-7B-Instruct and Qwen2.5-72B-Instruct, two sizes from the same family. Category B (reasoning LLM) is Qwen3-32B with thinking mode enabled; it emits an explicit reasoning trace before its answer. All three are dense transformers, not mixture-of-experts, so total and active parameter counts are equal. All models were accessed through the OpenRouter API (OpenAI-compatible chat-completions endpoint, `openai` Python SDK 3.26.0). For each model I pinned a single serving provider with fallbacks disabled (`provider.order=[...]`, `allow_fallbacks=false`, `require_parameters=true`), so every call for a model was served by the same provider and quantization. The served provider of every call was logged; 100% of calls were served by the pinned provider (Table 1). OpenRouter does not expose a revision hash, so the checkpoint is the official Hugging Face release that the OpenRouter model ID maps to.

Model	OpenRouter ID	Checkpoint	Parameters (total = active)	Category	Quantization	Provider (calls)
Qwen2.5-7B-Instruct	`qwen/qwen-2.5-7b-instruct`	`Qwen/Qwen2.5-7B-Instruct`	7.61B, dense	A	not reported by provider	Phala (1,034/1,034)
Qwen2.5-72B-Instruct	`qwen/qwen-2.5-72b-instruct`	`Qwen/Qwen2.5-72B-Instruct`	72.7B, dense	A	fp8	DeepInfra (1,034/1,034)
Qwen3-32B (thinking)	`qwen/qwen3-32b`	`Qwen/Qwen3-32B`	32.8B, dense	B	fp8	DeepInfra (1,034/1,034)

Table 1. Evaluated models. Quantization is the value reported by the provider on OpenRouter's endpoint listing. All access was API-based (OpenRouter).

Evaluation data. I use the Spider 1.0 development split (`dev.json`, `dev_gold.sql`) with its original SQLite databases (`database/`), not the test-suite databases. I evaluate on **all 1,034 dev examples**, in file order (example IDs 0–1033), so there is no subset sampling and no sampling seed. The difficulty labels (easy 248, medium 446, hard 174, extra 166) come from the official evaluation script's hardness function. The training split was not used: all prompts are zero-shot. As a sanity check, evaluating the gold SQL against itself gives EX = EM = 1.0.

Prompting and input. All models receive the same zero-shot chat prompt (prompt version `v1`):

````
[system]
You are an expert SQLite developer. Given a database schema and a question,
write one SQLite query that answers the question.

[user]
### Database schema
{schema}

### Question
{question}

Respond with the SQL query in a single ```sql fenced code block.
````

The schema is serialized from the SQLite file. Each table is rendered as a `CREATE TABLE` statement with column names and declared types, a `PRIMARY KEY (...)` clause and `FOREIGN KEY (...) REFERENCES ...` clauses. It is followed by a header row and 3 example rows (values truncated to 64 characters). An abbreviated example for `concert_singer`:

```
CREATE TABLE "singer" (
  "Singer_ID" INT, "Name" TEXT, "Country" TEXT, "Song_Name" TEXT,
  "Song_release_year" TEXT, "Age" INT, "Is_male" bool,
  PRIMARY KEY ("Singer_ID")
);
/* 3 example rows from "singer": */
Singer_ID  Name       Country      Song_Name  Song_release_year  Age  Is_male
1          Joe Sharp  Netherlands  You        1992               52   F
...
```

No few-shot examples are used. **SQL extraction** is deterministic and the same for all models:

1. Remove any `<think>...</think>` block (an unclosed `<think>` is removed to the end of the text).
2. Take the last non-empty ```` ```sql ```` fenced block (status `ok`).
3. Otherwise take the last `SELECT`/`WITH` statement in the text (status `fallback`).
4. Otherwise output the placeholder `SELECT 1 FROM __invalid__` (status `truncated` if the generation hit the length limit, else `missing`).

The extracted SQL is normalized to one line (whitespace collapsed, trailing semicolons removed). For Qwen3, the reasoning trace is returned separately by the API (`reasoning_details`), so the answer text contains only the final response.

Inference settings. Table 2 lists the decoding settings. Qwen2.5 models use greedy decoding (temperature 0). Qwen3-32B uses the sampling settings Qwen recommends for thinking mode, because greedy decoding is discouraged for it. Every request also sends `seed=0`; providers may not honour it, so the Qwen3 run should be treated as a single sample. Inference ran on the providers' remote hardware (GPU type not disclosed). The local machine only orchestrated requests, executed SQL and ran the evaluator: Intel Core i7-13620H (16 logical cores), 7.61 GiB RAM allocated to WSL2, Ubuntu 24.04.1 on WSL2, Python 3.12.3. Requests were sent with 8 concurrent threads, a 300 s timeout per request, and up to 6 attempts with exponential backoff on HTTP 429/5xx or empty responses.

Setting	Qwen2.5-7B / 72B	Qwen3-32B (thinking)
Decoding	greedy	sampling
Temperature	0.0	0.6
Top-p / top-k	not sent (irrelevant at T=0)	0.95 / 20
Max output tokens	512	16,384 (reasoning + answer)
Reasoning	off	on (`reasoning.enabled=true`, raw trace returned)
Seed	0	0

Table 2. Inference settings.

Before the full Qwen3 run, a preflight on the first 8 dev examples checked that the pinned provider returns non-empty, raw (not summarized) reasoning text with a plausible length, positive reasoning-token counts and extractable SQL, and that the served provider matched the pin. All 7 checks passed, so the full run proceeded.

Evaluation. Following the Spider paper and its official evaluator, I report **execution accuracy (EX)** and **exact-set match (EM)**, computed with the official `evaluation.py` from `taoyds/test-suite-sql-eval` . EX executes the predicted and gold SQL on the database and compares the results (row order matters only when the gold query has `ORDER BY`). EM parses both queries into Spider's clause structure and compares each clause as a set, ignoring literal values. EX is my primary metric. For per-example results I call the official functions directly, and on every run I check that the aggregated scores equal the official command-line output exactly (passed for all runs). Invalid outputs are counted as follows:

- No extractable SQL: replaced by the placeholder query, which fails to execute, so EX = 0 and EM = 0. This never happened (all 3,102 outputs extracted with status `ok`; Table 6).
- Non-executable SQL: EX = 0.
- **Unparseable SQL:** the 2018 Spider parser cannot parse some valid SQLite syntax, mainly column aliases (`COUNT(*) AS total`), table aliases without `AS` (`FROM singer s`), `INNER JOIN` and `CAST`. Following the official script, such predictions receive EM = 0. EX is unaffected because it does not use the parser. I report the number of unparseable predictions, EM on the parseable subset, and EX on the unparseable subset (Table 6).

For significance I report 95% percentile-bootstrap confidence intervals for overall EX (2,000 resamples, seed 0) and exact McNemar tests on paired per-example EX between models, Holm-adjusted across the three pairs (Table 4).

Results

Table 3 shows accuracy overall and by difficulty, and Figure 1 plots EX by difficulty.

Model	Easy (n=248)	Medium (n=446)	Hard (n=174)	Extra (n=166)	All (n=1,034)
Qwen2.5-7B-Instruct	87.1 / 52.0	80.5 / 28.3	55.7 / 22.4	51.8 / **6.6**	73.3 / 29.5
Qwen2.5-72B-Instruct	**90.3** / **70.6**	**87.9** / 30.3	**72.4** / **29.3**	**58.4** / **6.6**	**81.1** / **36.0**
Qwen3-32B (thinking)	89.9 / 61.7	82.7 / **34.5**	70.7 / 25.3	56.6 / 5.4	78.2 / 34.8

Table 3. EX / EM (%) on the full Spider dev set, overall and by difficulty. Best value per column in bold.

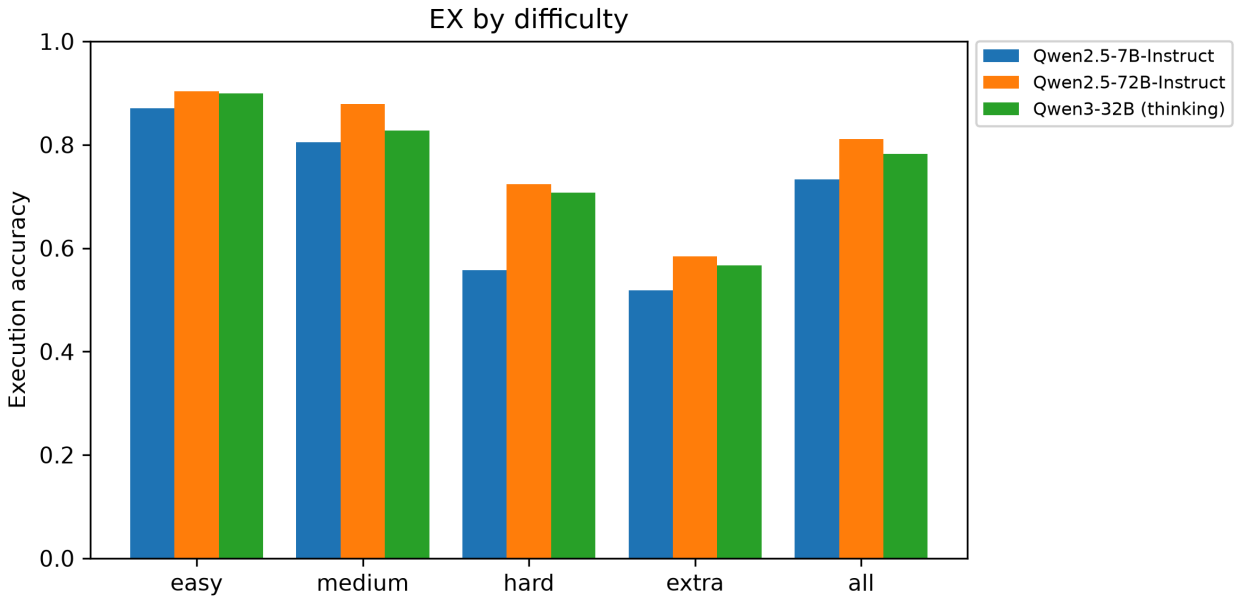


Figure 1. Execution accuracy by difficulty level for the three models.

Model	EX	95% CI	Comparison	Only first correct	Only second correct	p (Holm)
Qwen2.5-7B	73.3	[70.7, 76.1]	7B vs 72B	44	125	1.0e-9
Qwen2.5-72B	81.1	[78.7, 83.5]	7B vs Qwen3	68	119	4.7e-4
Qwen3-32B	78.2	[75.9, 80.7]	72B vs Qwen3	82	52	0.012

Table 4. Left: overall EX (%) with 95% bootstrap CIs. Right: pairwise exact McNemar tests on per-example EX.

Model	Mean latency (s)	Median / p90 latency (s)	Mean input tokens	Mean output tokens	Total output tokens	Run wall-clock (8 threads)
Qwen2.5-7B	0.57	0.51 / 0.87	837	37.9	39,186	1 min 34 s
Qwen2.5-72B	1.91	1.17 / 4.17	837	34.4	35,540	4 min 31 s
Qwen3-32B (thinking)	31.00	20.13 / 68.16	838	981.5	1,014,843	68 min 6 s

Table 5. Efficiency per example. Latency is wall time per API call, including network. Token counts are provider-reported `prompt_tokens`/`completion_tokens`; for Qwen3, output tokens include the reasoning trace (median 639, maximum 7,184 output tokens; no output hit the 16,384 limit). Monetary cost was not computed.

Model	Extraction ok	Non-executable	Unparseable by Spider parser	EM on parseable subset	EX on unparseable subset
Qwen2.5-7B	1,034 (100%)	49 (4.7%)	568 (54.9%)	65.5%	65.5%
Qwen2.5-72B	1,034 (100%)	10 (1.0%)	510 (49.3%)	71.0%	72.5%
Qwen3-32B	1,034 (100%)	6 (0.6%)	550 (53.2%)	74.4%	68.4%

Table 6. Invalid and unparseable outputs. No output needed the fallback or placeholder. Unparseable predictions receive EM = 0 from the official script.

Answer to RQ1. All three models handle cross-domain Text-to-SQL well on easy and medium questions but degrade sharply with complexity:

- **Scale is the strongest factor.** Qwen2.5-72B reaches the highest EX, 81.1% (Table 3). That is 7.8 points above Qwen2.5-7B (73.3%), with non-overlapping confidence intervals and p = 1.0e-9 (Table 4). The gap is largest on hard questions (72.4% vs 55.7%, +16.7 points). 72B also produces far fewer non-executable queries (1.0% vs 4.7%, Table 6).

- **Thinking helps a mid-size model, but does not beat the larger non-reasoning model.** Qwen3-32B with thinking is 4.9 points above the 7B model (78.2% vs 73.3%, p = 4.7e-4) and has the lowest non-executable rate (0.6%). It is still 2.9 points below Qwen2.5-72B (82 vs 52 discordant examples, p = 0.012). The gap comes mainly from medium questions (82.7% vs 87.9%).
- **Reasoning is expensive.** Qwen3 needs 16× the mean latency of 72B (31.0 s vs 1.91 s) and 29× the output tokens (981.5 vs 34.4) per example, and the full run took 68 minutes versus 4.5 minutes (Table 5).
- **Difficulty dominates.** EX drops from 87–90% on easy questions to 52–58% on extra-hard ones for every model (Figure 1). Extra-hard queries (nested subqueries, set operations, multiple joins) remain the main failure mode.
- **EM understates all models.** Overall EM is only 29.5–36.0%, because 49–55% of predictions use syntax the 2018 Spider parser cannot parse and are scored EM = 0 (Table 6). Many of these are correct: 65–73% of them have EX = 1. On parseable predictions EM is 65.5–74.4%. I therefore base my conclusions on EX. EM is mainly informative about whether a model's SQL style matches Spider's.

Limitations: Qwen3 was sampled once at temperature 0.6, so its EX has sampling variance beyond the bootstrap CI. The fp8 serving and the unreported quantization for 7B may differ slightly from bf16 results. EX on the original databases can occasionally count a semantically wrong query as correct.

2. Task 2 – RQ2: How do characteristics of reasoning traces relate to Text-to-SQL performance?

Methodology

Data. I analyze the reasoning traces of Qwen3-32B (thinking) from Task 1: all 1,034 traces, one per dev example. The raw trace text came from the API's `reasoning_details` field. No trace was empty or missing, so nothing was excluded. Correctness is the Task 1 EX label (809 correct, 225 incorrect); difficulty is the official label.

Features. I extract seven features from every trace (Table 7). Trace length is measured by tokenizing the trace text with the Qwen3 tokenizer (`Qwen/Qwen3-32B`, Hugging Face `tokenizers` 0.23.2) rather than using the provider's `reasoning_tokens` field. That field is an estimate on DeepInfra: it exceeded the total `completion_tokens` in 775 of 1,034 calls, which is impossible for a true count. The text-based count is never larger than `completion_tokens` and correlates almost perfectly with the provider figure (Spearman 0.994; mean 937.1 vs 1,023.6 tokens). Character length is reported as a tokenizer-independent robustness check.

Feature	Definition and computation	Why it is relevant to Text-to-SQL
`length_tokens`	Number of Qwen3 tokens in the trace text	Measures how much deliberation the model needed; long traces may signal hard questions or uncertainty about schema/joins
`length_chars`	Number of characters in the trace	Tokenizer-independent check of the length result
`marker_count`	Case-insensitive, word-bounded matches of the self-correction markers *wait, actually, let me check, double-check, alternatively, hmm*	Counts explicit revisions of the plan (e.g., reconsidering a join or condition)
`markers_per_1k`	`marker_count / length_tokens × 1000`	Hesitation density, separated from trace length
`sql_drafts`	Number of candidate SQL statements in the trace: fenced ```` ```sql ````/bare fenced blocks containing `SELECT`/`WITH`, plus uppercase inline `SELECT ... FROM` / `WITH x AS (` statements outside fences	How many alternative queries the model wrote before answering; indicates search over query structures
`table_coverage`	Fraction of gold-query tables whose names appear in the trace (case-insensitive, word-bounded)	Whether the model attended to the tables the answer needs (schema linking)
`column_coverage`	Same for gold-query columns (* excluded); undefined for the 42 gold queries that reference no column (e.g., `SELECT count(*) FROM t`)	Whether the model found the right columns; column choice is a common Text-to-SQL error

Table 7. Trace features. Gold tables and columns are obtained by parsing the gold SQL with the official Spider parser and mapping to original schema names.

Statistical tests. The features are counts and ratios with strongly right-skewed, heavy-tailed distributions (Figure 2) and many ties, and the coverage features sit at a ceiling of 1.0. I therefore use rank-based tests:

- **Correctness:** Mann–Whitney U test (correct vs incorrect), which compares two independent groups without assuming normality. The effect size is the rank-biserial correlation $r = 2U/(n_1 n_2) - 1$, which is positive when correct traces have larger values.
- **Difficulty:** Kruskal–Wallis H test across the four difficulty levels, a non-parametric one-way test for more than two groups. Because difficulty is ordinal, I add Spearman's ρ between each feature and difficulty coded easy = 1 … extra = 4, which measures monotonic trends.
- **Multiple testing:** within each test family (7 features), p-values are Holm-adjusted.
- **Joint model:** harder questions are both longer and more often wrong, so a feature could relate to correctness only through difficulty. I therefore fit a **logistic regression** of EX (binary outcome) on z-scored `length_tokens`, `markers_per_1k`, `sql_drafts` and `column_coverage`, plus difficulty as a categorical covariate (reference: easy), by maximum likelihood. `table_coverage` is omitted because it equals 1.0 for all but one trace, which causes quasi-separation. `length_chars` and `marker_count` are omitted because they are collinear with `length_tokens`. The fit uses the 992 traces with a defined `column_coverage`, and the model converged.

Results

Tables 8 and 9 give descriptive statistics by correctness and by difficulty, Table 10 the rank tests, and Table 11 the logistic regression. Figures 2 and 3 show box plots of every feature.

Feature	Correct (n=809): median [Q1, Q3]	mean	Incorrect (n=225): median [Q1, Q3]	mean
length_tokens	517 [356, 885]	778.8	1,189 [696, 1,929]	1,506.3
length_chars	2,271 [1,586, 3,912]	3,367.5	5,046 [3,186, 8,361]	6,465.7
marker_count	4 [2, 7]	4.96	8 [5, 11]	8.44
markers_per_1k	6.62 [4.97, 8.66]	6.91	6.25 [4.46, 7.89]	6.47
sql_drafts	1 [1, 3]	2.91	3 [1, 5]	5.30
table_coverage	1 [1, 1]	1.000	1 [1, 1]	0.999
column_coverage*	1 [1, 1]	0.996	1 [1, 1]	0.970

*Table 8. Trace features by EX correctness. *column_coverage: n = 769 correct, 223 incorrect.*

Feature	Easy (n=248)	Medium (n=446)	Hard (n=174)	Extra (n=166)
length_tokens	362 [279, 555]	504 [381, 805]	1,073 [638, 1,902]	1,268 [850, 1,991]
length_chars	1,629 [1,262, 2,433]	2,240 [1,705, 3,477]	4,432 [2,715, 8,004]	5,496 [3,679, 8,742]
marker_count	3 [2, 5]	4 [2, 6]	7 [4, 10]	8 [5, 12]
markers_per_1k	7.14 [5.00, 9.05]	6.95 [5.12, 9.01]	6.06 [4.80, 7.76]	5.66 [4.43, 7.26]
sql_drafts	1 [1, 2]	1 [1, 2]	4 [2, 8]	4 [2, 7]
table_coverage (mean)	1.000	1.000	1.000	0.998
column_coverage (mean)*	0.987	0.996	0.987	0.982

*Table 9. Trace features by difficulty: median [Q1, Q3] unless marked. *column_coverage easy n = 206 (the 42 undefined cases are all easy).*

Feature	Mann–Whitney r (correct vs incorrect)	p (Holm)	Kruskal–Wallis H	p (Holm)	Spearman ρ with difficulty	p (Holm)
length_tokens	−0.501	7.8e-30	331.6	1.0e-70	0.560	1.9e-85
length_chars	−0.505	2.9e-30	323.7	4.5e-69	0.552	8.1e-83
marker_count	−0.455	3.6e-25	220.0	7.9e-47	0.455	3.7e-53
markers_per_1k	+0.100	0.043	32.2	1.4e-6	−0.156	1.5e-6
sql_drafts	−0.309	1.3e-13	236.8	2.3e-50	0.440	1.0e-49
table_coverage	+0.004	0.058	5.2	0.156	−0.048	0.125
column_coverage	+0.051	1.5e-4	18.3	7.5e-4	−0.088	0.011

Table 10. Rank tests. Negative r means incorrect traces have larger values. Holm adjustment is within each test family.

Term	Odds ratio	95% CI	p
Difficulty: medium (vs easy)	0.66	[0.39, 1.11]	0.116
Difficulty: hard (vs easy)	0.45	[0.25, 0.82]	0.009
Difficulty: extra (vs easy)	0.29	[0.16, 0.51]	2.9e-5
length_tokens (per SD)	**0.40**	[0.31, 0.51]	7.2e-13
markers_per_1k (per SD)	0.86	[0.71, 1.03]	0.100
sql_drafts (per SD)	**1.50**	[1.19, 1.90]	5.7e-4
column_coverage (per SD)	**1.39**	[1.17, 1.64]	1.1e-4

Table 11. Logistic regression of EX on z-scored trace features and difficulty (maximum likelihood, n = 992, converged). Odds ratio < 1 means lower odds of a correct answer.

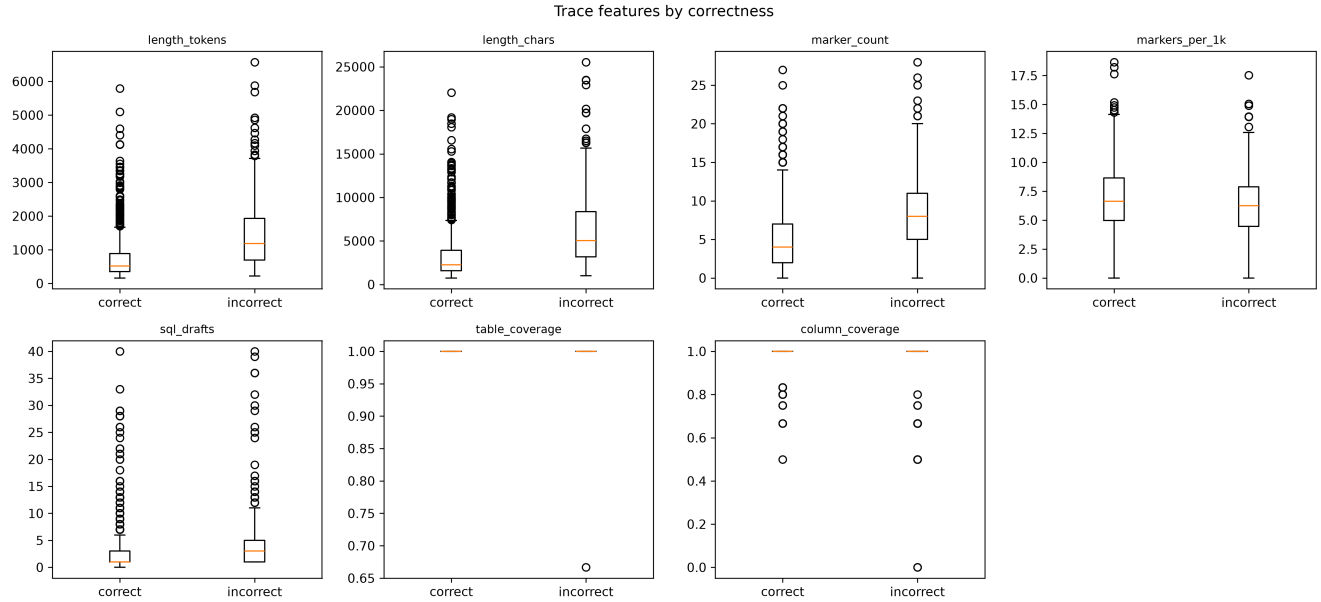


Figure 2. Box plots of each trace feature by EX correctness (Qwen3-32B, n = 1,034).

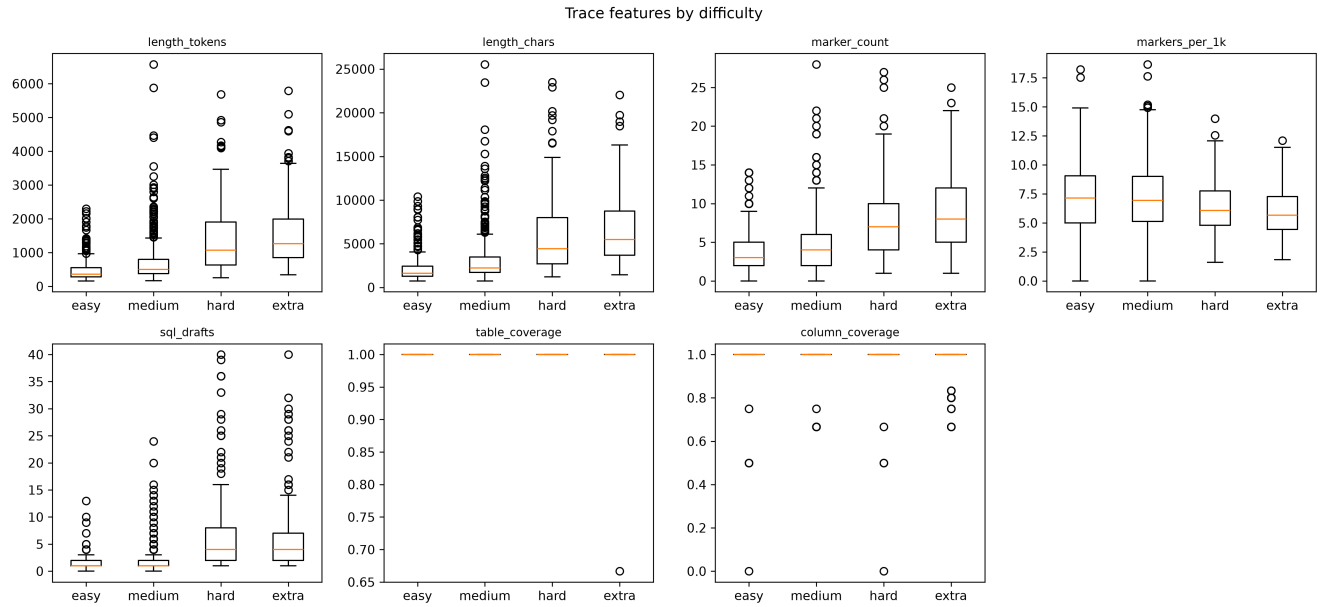


Figure 3. Box plots of each trace feature by difficulty level.

Answer to RQ2.

- **Length is the strongest correlate of failure.** Incorrect answers come with traces more than twice as long as correct ones (median 1,189 vs 517 tokens; Table 8), a large effect (r = −0.50, p = 7.8e-30; Table 10). The character count gives the same result (r = −0.505), so the finding does not depend on the tokenizer.
- **Length tracks difficulty, but difficulty does not explain it away.** Median length grows from 362 tokens (easy) to 1,268 (extra) (ρ = 0.56; Table 9). In the logistic regression, which controls for difficulty, each standard deviation of extra length still multiplies the odds of a correct answer by 0.40 (95% CI [0.31, 0.51]; Table 11). Within the same difficulty level, a long trace is a warning sign. It most likely reflects the model being uncertain (e.g., about joins or ambiguous columns), not length causing errors.
- **Self-correction markers scale with length, not with failure.** Incorrect traces contain more markers (median 8 vs 4), but per 1,000 tokens they contain slightly fewer (6.25 vs 6.62; r = +0.10, p = 0.043). The density also falls with difficulty (ρ = −0.16). After controlling for length and difficulty, marker density is not

significant (OR 0.86, p = 0.10). Words like “wait” therefore do not indicate a failing trace beyond what length already shows.

- **Drafting more queries is not bad in itself.** Incorrect and hard questions have more SQL drafts (median 3 vs 1; 4 for hard/extra vs 1 for easy/medium). Holding length fixed, more drafts goes with *higher* odds of success (OR 1.50, p = 5.7e-4). Long traces that keep writing concrete candidate queries do better than long traces without them. This is a conditional association between correlated features and should be read cautiously.
- **Schema coverage is near ceiling.** The trace mentions every gold table in all but one case (Table 8; differences not significant). Column coverage is also near 1, but slightly lower for incorrect answers (0.970 vs 0.996 mean; OR 1.39 per SD, p = 1.1e-4). Qwen3 almost always finds the right tables; its errors are mostly about query logic, with an occasional missed column.

3. Task 3 – RQ3: To what extent does Execution-Guided Self-Correction (EGSC) improve Qwen2.5-7B-Instruct?

Methodology

Model choice. I improve the Qwen2.5-7B-Instruct solution from Task 1. It is the cheapest and fastest model (0.57 s per query; Table 5) but the least accurate (73.3% EX). It also has by far the most non-executable outputs: 49 (4.7%), versus 10 for 72B and 6 for Qwen3 (Table 6). Most of these failures are schema-grounding errors: 32 of the 49 are `no such column` and 4 are `ambiguous column name`. Another 8 use functions SQLite lacks (e.g., `YEAR()`), and 5 are other errors (wrong argument count, unrecognized token, unknown table, syntax). Such errors produce concrete, free feedback from the database at inference time. Improving a small model without retraining is practically interesting because it keeps the low cost and latency.

Strategy: Execution-Guided Self-Correction (EGSC), an Option A strategy that improves how the model reasons about its own query using database feedback. Model parameters are not changed. For each dev example:

1. Execute the Task 1 prediction of Qwen2.5-7B (unchanged) on the database, with a 5 s timeout.
2. If it raises an **execution error or times out**, send the model the original conversation (same system prompt and schema/question prompt as Task 1), then its failed query as an assistant turn, then a user turn: *“The query above failed when executed on the database. Execution feedback: {error message}. Fix the query. Respond with the corrected SQL in a single ```sql fenced code block.”*
3. Extract and execute the new query, repeating up to **2 retries**.
4. The final prediction is the last attempt that executed successfully. If none did, it is the original query.

Queries that execute are never changed, so EGSC can only alter the 49 failing queries. The **gold SQL is never used**: only the question, schema and execution result enter the loop. Gold is used only for scoring. The retries use the same model and decoding as Task 1 (temperature 0, max 512 tokens, seed 0, extraction rules unchanged). The retry calls were not explicitly pinned to a provider, and their provider was not logged; Phala was the only OpenRouter endpoint listed for this model. I expected EGSC to make most failing queries executable and convert a share of them into correct answers. Its gain is capped at 4.7 points, and it should help most on hard/extra questions, where 31 of the 49 failures occur.

Evaluation. I use the same 1,034 dev examples, the same official evaluator, the same EX/EM metrics and the same invalid-output rules as Task 1. I report results overall and by difficulty, and compare with the original 7B solution using an exact McNemar test on paired per-example EX.

Results

System	Easy	Medium	Hard	Extra	All	Non-exec.	Unparseable	EM on parseable
Qwen2.5-7B (original, Task 1)	87.1 / 52.0	80.5 / 28.3	55.7 / 22.4	51.8 / 6.6	73.3 / 29.5	49	568	65.5
Qwen2.5-7B + EGSC	87.5 / 52.0	80.9 / 28.3	59.2 / 23.6	54.2 / 7.2	**74.6** / **29.8**	10	564	65.5
Δ EX (points)	+0.4	+0.4	+3.4	+2.4	**+1.3**			

Table 12. Original vs EGSC on the full Spider dev set: EX / EM (%) by difficulty, number of non-executable and parser-unparseable predictions, and EM (%) on the parseable subset.

Triggered	Fixed (0→1)	Broken (1→0)	Unchanged	McNemar p	Extra LLM calls	Extra prompt / completion tokens	Extra LLM time
49 (4.7%)	13	0	1,021	2.4e-4	65 (+6.3%)	65,569 (+7.6%) / 3,872 (+9.9%)	39.8 s (+6.7%)

Table 13. EGSC counts, exact McNemar test on paired EX, and overhead relative to the original run (1,034 calls, 865,432 prompt and 39,186 completion tokens, about 592 s of LLM time).

Answer to RQ3. EGSC improves Qwen2.5-7B-Instruct by **+1.3 EX points** (73.3% → 74.6%; Table 12). The gain is small but statistically significant: 13 examples fixed and none broken (exact McNemar p = 2.4e-4; Table 13). It costs 6.3% more LLM calls. EM barely changes (29.5% → 29.8%) because the parser limitation from Task 1 still applies.

- **Partly confirmed: executability.** Of the 49 triggered queries, 39 (80%) executed after correction, so non-executable outputs fell from 49 to 10. 33 needed one retry and 16 needed two. As expected, the gains concentrate on hard (+3.4 points, 6 fixed) and extra-hard (+2.4 points, 4 fixed) questions. Easy and medium gained 1 and 2 examples respectively.
- **Not confirmed: correctness.** Only 13 of the 49 (27%) became correct. The other 26 executable repairs still returned wrong results. The error message tells the model *that* a column does not exist, not which join path or condition the question needs. The model often "fixes" the error by restructuring the query in a way that runs but is semantically wrong.
- **Limited reach.** The strategy only touches the 4.7% of queries that fail to execute. Of the 276 questions 7B gets wrong, 227 execute without error, so EGSC never sees them. EGSC closes about 16% of the 7.8-point gap to Qwen2.5-72B (74.6% vs 81.1%).

Examples.

Fixed (example 103, `car_1`, hard). "Which distinct car models are produced after 1980?" The original query `SELECT DISTINCT T2.Model FROM cars_data AS T1 JOIN model_list AS T2 ON T1.MakeId = T2.ModelId WHERE T1.Year > 1980` failed with `no such column: T1.MakeId`. With that feedback the model found the correct join path through `car_names`: `SELECT DISTINCT T3.Model FROM cars_data AS T1 JOIN car_names AS T2 ON T1.Id = T2.MakeId JOIN model_list AS T3 ON T2.Model = T3.Model WHERE T1.Year > 1980`. This matches the gold result.

Fixed (example 374, `cre_Doc_Template_Mgt`, extra). "Return the id and name of the document with the most paragraphs." The original failed with `ambiguous column name: Document_ID`. The retry qualified every column with table aliases (`D.Document_ID, D.Document_Name ... GROUP BY D.Document_ID, D.Document_Name ORDER BY COUNT(P.Paragraph_ID) DESC LIMIT 1`) and became correct.

Executable but still wrong (example 119, `car_1`, medium). "What is the smallest weight of the car produced with 8 cylinders on 1974?" The original used `t2.Year` on the wrong table. After two retries the query executed, but the model had added an unnecessary join and an invented filter (`t3.Make = 'plymouth satellite'`). The gold query is simply `SELECT min(weight) FROM cars_data WHERE cylinders = 8 AND year = 1974`. This illustrates the main limitation: execution feedback rewards running queries, not correct ones. No example was broken, because EGSC never changes a query that already executes.

4. Task 4 – AI Usage Declaration

I used Kiro, an AI-powered IDE agent built on Anthropic Claude models, throughout this assignment. Kiro helped plan the study design (model selection, metric choice, statistical tests), help me debugged the Python pipeline and notebooks (OpenRouter client, schema serialization, prompt and SQL extraction, wrapper around the official Spider evaluator, trace-feature extraction, statistics, and the EGSC loop), wrote the unit tests, diagnosed issues in intermediate runs (e.g., provider-reported reasoning-token counts exceeding completion tokens, and the official parser rejecting column aliases), and drafted this report from the generated result tables. I made the main decisions (models, budget, zero-shot prompting, Task 3 strategy), write the spec and requirement for kiro to run (since kiro is spec-based) ran all experiments myself, and checked the output by: running the unit test suite; verifying on every run that my per-example evaluation reproduces the official `evaluation.py` command-line totals exactly; running small-scale (10-example) trial runs before the full runs; inspecting raw model outputs, reasoning traces and EGSC examples by hand; and checking every number in this report against the generated result files. I take full responsibility for the correctness of the code, results and claims.